
Working with Relationships in Power BI

In this tutorial, you will learn how relationships work in Power BI Desktop.

Step 1

Click on Data View, your model should look like the one below. This model has four tables, a fact table (ProjectBudget) and three dimensions tables (CompanyEmployee, ProjectHours and CompanyProject). **Note that there is no connection (relationship) between the tables.**

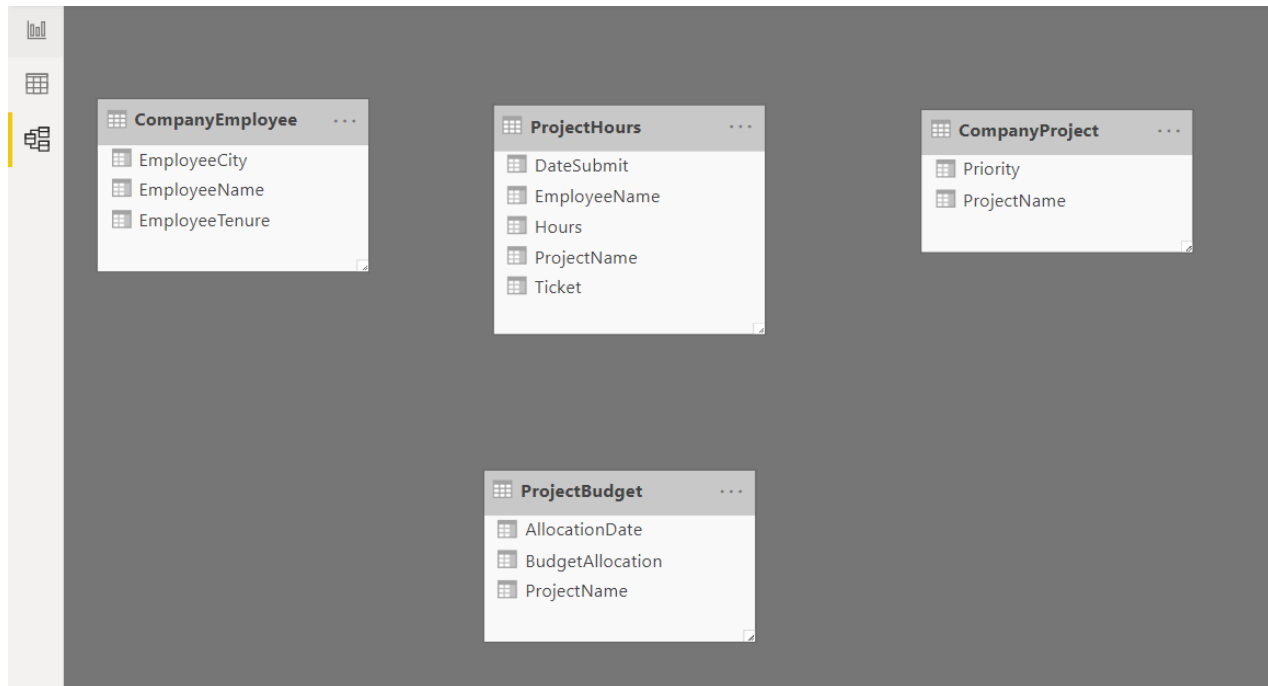


Table	Fields
CompanyEmployee	EmployeeCity, EmployeeName, EmployeeTenure
ProjectHours	DateSubmit, EmployeeName, Hours, ProjectName, Ticket
CompanyProject	Priority, ProjectName
ProjectBudget	AllocationDate, BudgetAllocation, ProjectName

Let's have a closer look at these table.

Click on Transform Data (Edit Queries in the old version) in the Home ribbon, to open the Query Editor. Have a look at the four tables by selecting them using the menu on the left.

BusinessDataModel - Power Query Editor

File Home Transform Add Column View Tools Help

Close & Apply New Source Recent Sources Enter Data Data source settings Manage Parameters Refresh Preview Properties Advanced Editor Manage Choose Columns Remove Columns Keep Rows Remove Rows Sort Split Column Group By Data Type: Whole Number Use First Row as Headers Replace Values

Queries [4]

- ProjectHours
- ProjectBudget
- CompanyProject
- CompanyEmployee

= Table.TransformColumnTypes("#Promoted Headers",{{"Ticket", Int64.Type}, {"EmployeeName", type text}, {"Ho

	Ticket	EmployeeName	Hours	ProjectName	DateSubmit
1	1001	Brewer, Alan	22	Blue	01/01/2013
2	1002	Brewer, Alan	26	Red	02/01/2013
3	1004	Brewer, Alan	13	Orange	01/02/2012
4	1005	Bowen, Eli	29	Purple	10/01/2013
5	1003	Ito, Shu	34	Yellow	12/04/2012
6	1006	Bento, Nuno	35	Green	02/01/2013
7	1007	Hamilton, David	10	Yellow	10/01/2013
8	1008	Han, Mu	28	Orange	01/02/2012
9	1009	Ito, Shu	22	Purple	02/01/2013
10	1010	Bowen, Eli	28	Green	10/01/2013
11	1011	Bowen, Eli	9	Blue	10/12/2013

ProjectHours. This table is a record of the tickets that record the number of hours to complete a particular project. You can also see the name of the Employee who submitted the project, the name of the project and the submission date.

The **ProjectBudget** table below includes the name of the project, the budget allocated to that project and the date when the project has been allocated.

BusinessDataModel - Power Query Editor

File Home Transform Add Column View Tools Help

Close & Apply New Source Recent Sources Enter Data Data source settings Manage Parameters Refresh Preview Properties Advanced Editor Manage Choose Columns Remove Columns Keep Rows Remove Rows

Queries [4]

- ProjectHours
- ProjectBudget**
- CompanyProject
- CompanyEmployee

`= Table.TransformColumnTypes(#"Promoted Headers",{{"ProjectName", type text, "BudgetAllocation", type number, "AllocationDate", type date}}`

	ProjectName	BudgetAllocation	AllocationDate
1	Blue	40000	12/01/2012
2	Red	100000	12/01/2012
3	Green	50000	12/01/2012

The **CompanyProject** table below is a list of the projects with an assigned priority: A, B, or C.

Queries [4] < ✕ ✓ fx = Table.TransformColumnTypes(#"Promot

	A ^B _C ProjectName	A ^B _C Priority
1	Blue	A
2	Red	B
3	Orange	C
4	Purple	B
5	Yellow	C
6	Green	C

Finally, the **CompanyEmployee** table shows the name of each employee of the company, the city of the employee and their tenure.

Queries [4] < ✕ ✓ fx = Table.TransformColumnTypes(#"Promoted Headers",{{"Employee

	A ^B _C EmployeeName	A ^B _C EmployeeCity	1 ² ₃ EmployeeTenure
1	Brewer, Alan	Redmond	15
2	Bowen, Eli	San Jose	10
3	Ito, Shu	Redmond	1
4	Bento, Nuno	Redmond	15
5	Hamilton, David	San Jose	3
6	Han, Mu	San Jose	1

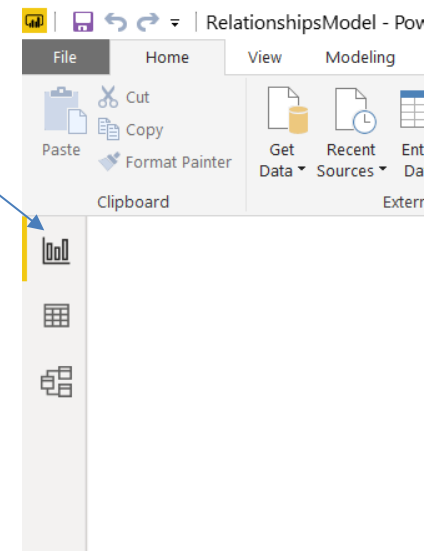
For the first part of this tutorial we will focus on the **CompanyProject** table (on the left) and the **ProjectHours** table (on the right).

	ProjectName	Priority
1	Blue	A
2	Red	B
3	Orange	C
4	Purple	B
5	Yellow	C
6	Green	C

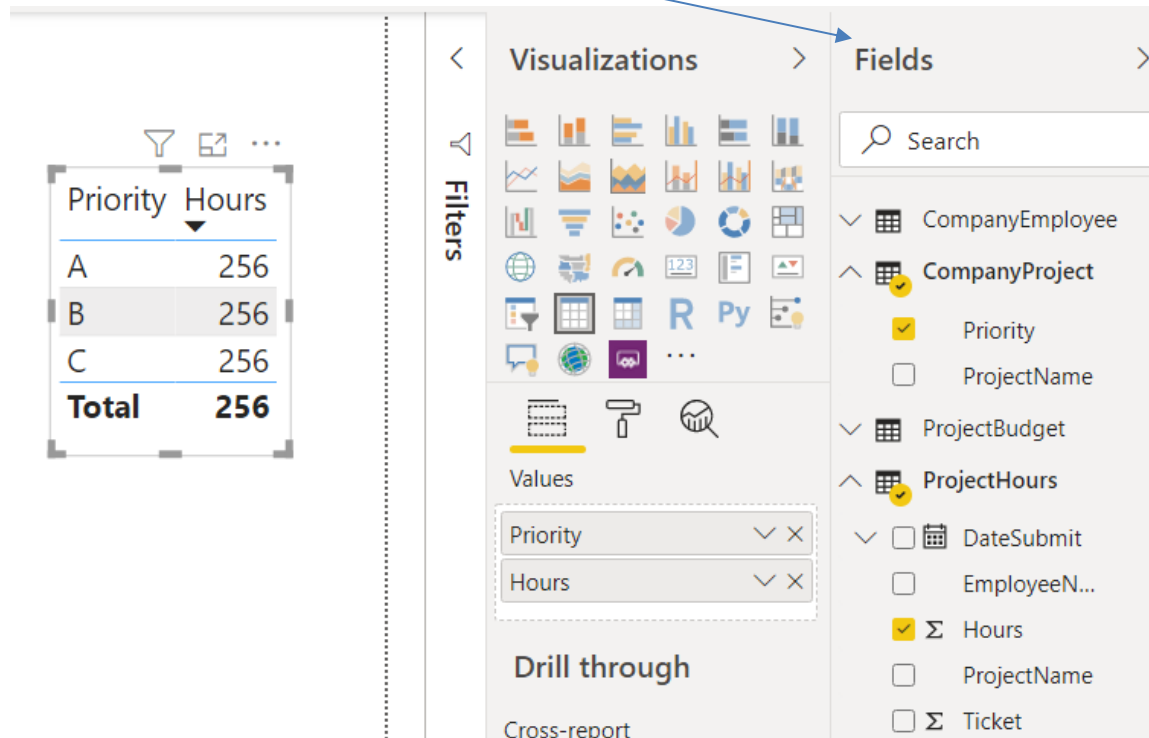
	Ticket	EmployeeName	Hours	ProjectName	DateSubmit
1	1001	Brewer, Alan	22	Blue	01/01/2013
2	1002	Brewer, Alan	26	Red	02/01/2013
3	1004	Brewer, Alan	13	Orange	01/02/2012
4	1005	Bowen, Eli	29	Purple	10/01/2013
5	1003	Ito, Shu	34	Yellow	12/04/2012
6	1006	Bento, Nuno	35	Green	02/01/2013
7	1007	Hamilton, David	10	Yellow	10/01/2013
8	1008	Han, Mu	28	Orange	01/02/2012
9	1009	Ito, Shu	22	Purple	02/01/2013
10	1010	Bowen, Eli	28	Green	10/01/2013
11	1011	Bowen, Eli	9	Blue	10/12/2013

Notice that in both table there is a ProjectName column. That is important, and we'll get back to it in soon.

Now we are going to use these two table to create a simple report. Click on the report view.



The first thing we want to get is the number of hours submitted by project priority. To do so, in the Fields section on the right of the report view, tick the box next to the **Priority** column from the **CompanyProject** table and the **Hours** column from the **ProjectHours** table (as shown below). You will see a table in the Report canvas. If you can't see the columns' for each table, click on the drop down arrow next to each table to expand them.



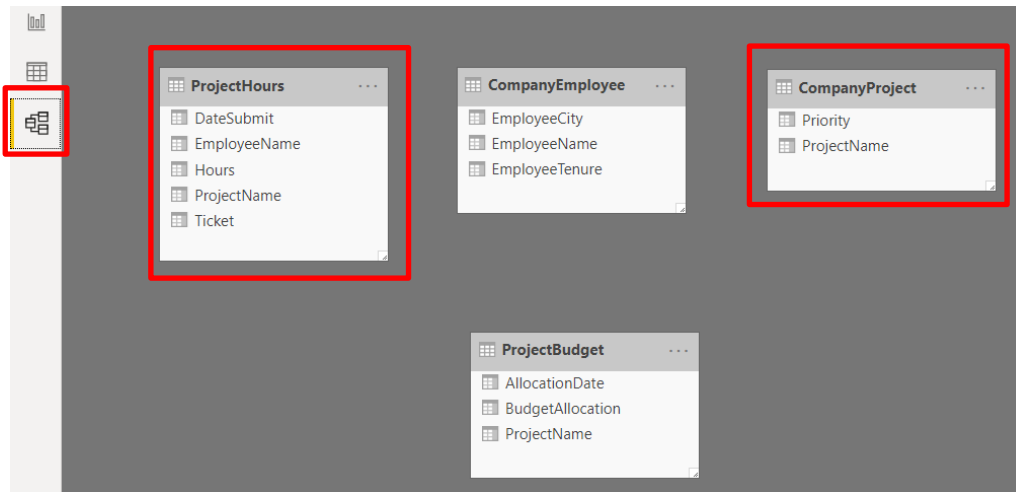
The screenshot shows a report builder interface. On the left, a table is displayed in the report canvas with the following data:

Priority	Hours
A	256
B	256
C	256
Total	256

On the right, the 'Fields' panel is visible, showing a list of tables and their columns. The 'CompanyProject' table is expanded, showing 'Priority' and 'ProjectName'. The 'ProjectHours' table is also expanded, showing 'DateSubmit', 'EmployeeN...', 'Hours', 'ProjectName', and 'Ticket'. The 'Hours' column from 'ProjectHours' is selected with a checkmark and a summation symbol (Σ).

If we look at our table in the report canvas, you will see the number of hours is 256 for each project, which is also the total. Clearly, this number is not correct. Why? It is because we cannot calculate the total of values **Hours** from one table (the **Project** table), sliced by the values **Priority** in another table (the **CompanyProject** table) without a relationship between these two tables. In fact, if

you open the model view tab, you will see that there is no relationship between the tables, as shown below.



So, let's create a relationship between these two tables. Remember the two **ProjectName** columns we saw in both tables with the project name? We will use these two columns to create a relationship between our tables.

Why these columns? Well, if we look at the **ProjectName** column in the **ProjectHours** table (on the right below), we see values like Blue, Red, Yellow, Orange, and so on. In fact, we see several rows that have the same value. In effect, we have many colour values for **ProjectName** (this will be the "many" table in a many-to-one relationship).

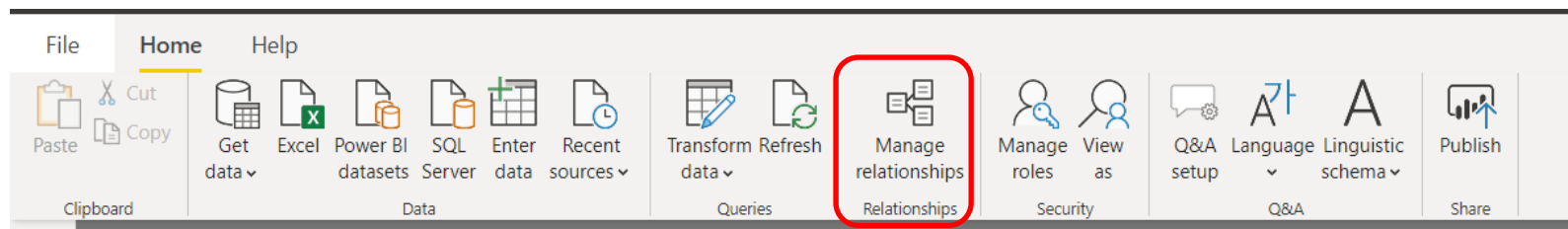
	ProjectName	Priority
1	Blue	A
2	Red	B
3	Orange	C
4	Purple	B
5	Yellow	C
6	Green	C

	Ticket	EmployeeName	Hours	ProjectName	DateSubmit
1	1001	Brewer, Alan	22	Blue	01/01/2013
2	1002	Brewer, Alan	26	Red	02/01/2013
3	1004	Brewer, Alan	13	Orange	01/02/2012
4	1005	Bowen, Eli	29	Purple	10/01/2013
5	1003	Ito, Shu	34	Yellow	12/04/2012
6	1006	Bento, Nuno	35	Green	02/01/2013
7	1007	Hamilton, David	10	Yellow	10/01/2013
8	1008	Han, Mu	28	Orange	01/02/2012
9	1009	Ito, Shu	22	Purple	02/01/2013
10	1010	Bowen, Eli	28	Green	10/01/2013
11	1011	Bowen, Eli	9	Blue	10/12/2013

If we look at the **ProjectName** column in the **CompanyProject** table (on the left above), we see that there is only one of each of the colour values for the project name (this will be the “one” table in a many-to-one relationship). Each colour value in this table is unique, and that is important, because we can create a relationship between these two tables. In this case, a many-to-one relationship. In a many-to-one relationship, at least one column in one of the tables must contain unique values (the CompanyProject table has unique values for ProjectNames). There are some additional options for some relationships, which we will look at later. For now, let’s create a relationship between the project columns in each of our two tables.

To create the new relationship

1. Select **Manage Relationships** from the **Home** ribbon (make sure you are in the Model View).



2. In **Manage relationships**, select **New** to open the **Create relationship** dialog box, where we can select the tables, columns, and any additional settings we want for our relationship.
3. In the first drop-down list, select **ProjectHours** as the first table (see image below), then select the **ProjectName** column by clicking on it. This side is the *many* side of our relationship.
4. In the second drop-down list, select **CompanyProject** (see image below). Click on the **ProjectName** column to select it. This side is the *one* side of our relationship.
5. Accept the defaults for the relationship options, and then select **OK**.

Make sure you selected the columns by clicking on them.

Create relationship



Select tables and columns that are related.

ProjectHours

Ticket	EmployeeName	Hours	ProjectName	DateSubmit
1001	Brewer, Alan	22	Blue	01 January 2013
1002	Brewer, Alan	26	Red	02 January 2013
1004	Brewer, Alan	13	Orange	01 February 2012

CompanyProject

ProjectName	Priority
Blue	A
Red	B
Orange	C

Cardinality

Many to one (*:1)

Cross filter direction

Single

☒ Make this relationship active

☐ Apply security filter in both directions

☐ Assume referential integrity

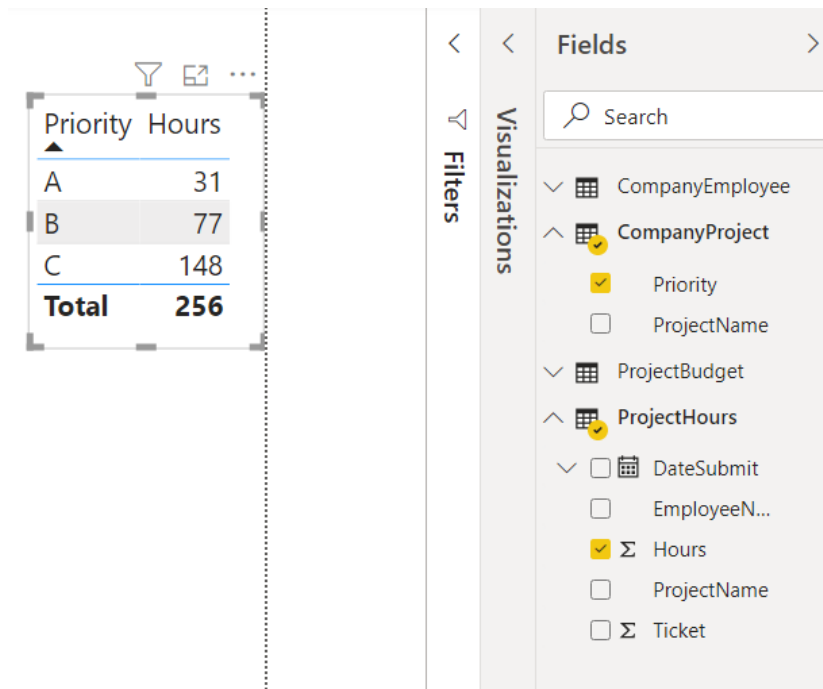
OK

Cancel

6. In the **Manage relationships** dialog box, select **Close** at the bottom right corner.

In the interest of full disclosure, you just created this relationship the hard way. You could have just selected **Autodetect** in the **Manage relationships** dialog box that opened in Step 2. In fact, autodetect could have automatically created the relationship for you when you loaded the data if both columns had the same name. But, what's the challenge in that?

Now, let's look at the table in our report canvas again. Go back to Report View and check your table.

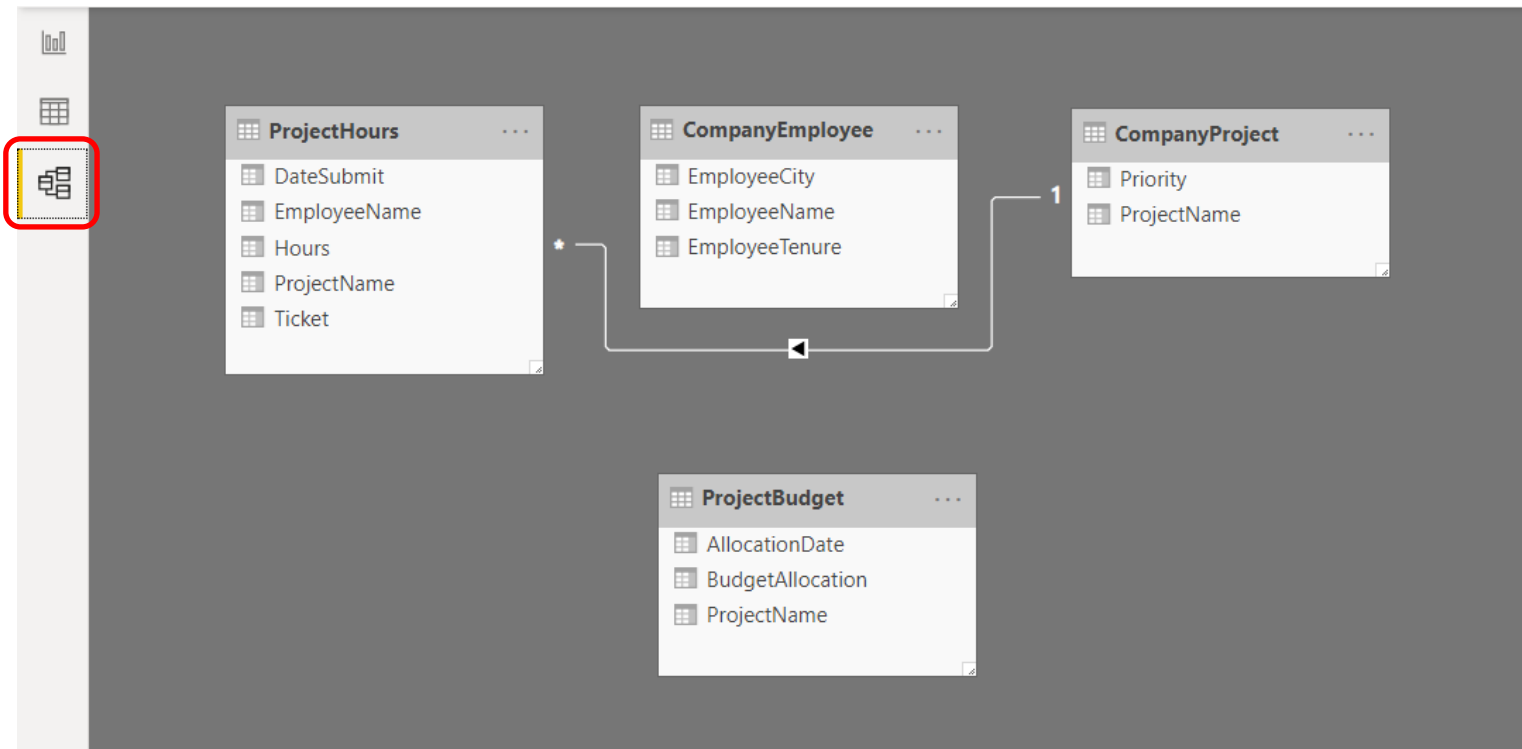


The screenshot shows a Power BI Desktop interface. On the left, a table visualization displays data with columns 'Priority' and 'Hours'. The table has three rows: 'A' with 31 hours, 'B' with 77 hours, and 'C' with 148 hours. A 'Total' row at the bottom shows a sum of 256 hours. On the right, the 'Fields' pane is visible, showing a list of tables and their columns. The 'CompanyProject' table is expanded, showing 'Priority' and 'ProjectName'. The 'ProjectHours' table is also expanded, showing 'DateSubmit', 'EmployeeN...', 'Hours', 'ProjectName', and 'Ticket'. The 'Hours' column in 'ProjectHours' is selected, indicated by a yellow checkmark and a summation symbol (Σ).

Priority	Hours
A	31
B	77
C	148
Total	256

That looks a whole lot better, doesn't it?

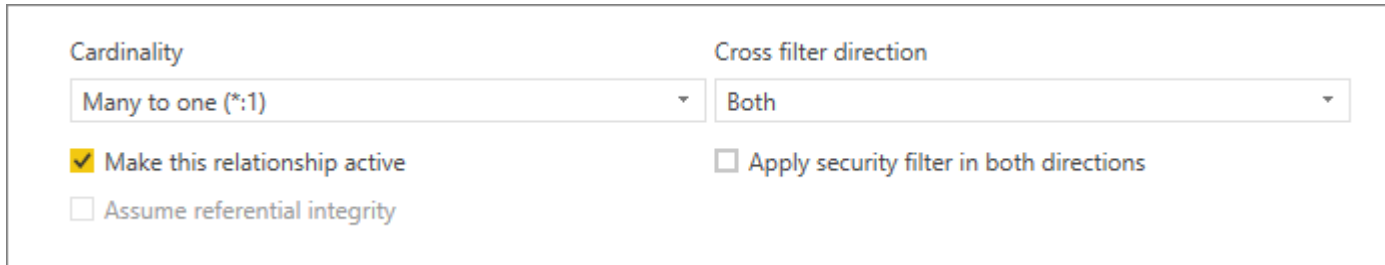
When we sum up hours by **Priority**, Power BI Desktop looks for every instance of the unique colour values in the **CompanyProject** lookup table and matches every instance of each of those values to the values in the **ProjectHours** table, and then calculates a total for each priority value. This is possible only because you created a relationship before. You can see the new relationship between the tables in Model view



Well done! You have just created your first relationship in Power BI!

Understanding additional options

When a relationship is created, either with autodetect or manually, Power BI Desktop automatically configures additional options based on the data in your tables. These additional relationship options are located in the lower portion of the **Create relationship** and **Edit relationship** dialog boxes.

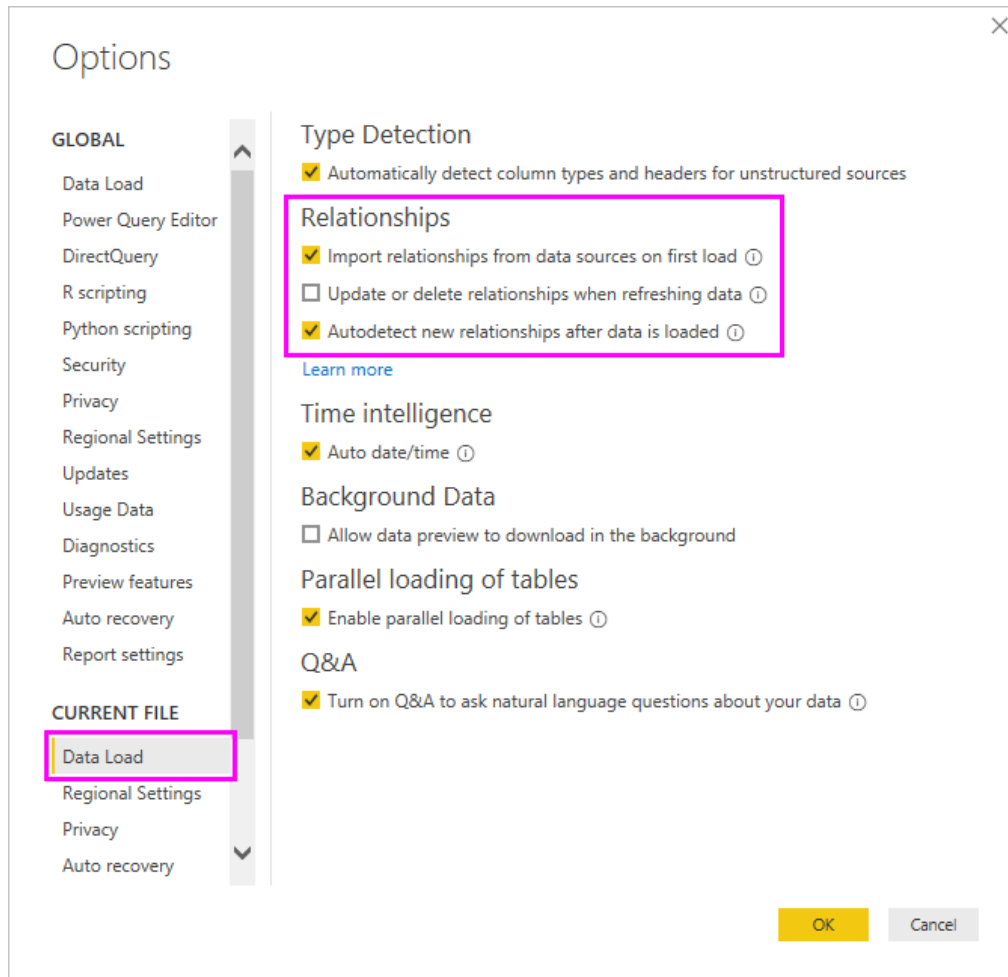


The screenshot shows a dialog box with two columns of options. The first column is titled 'Cardinality' and contains a dropdown menu set to 'Many to one (*:1)'. Below it are two checkboxes: 'Make this relationship active' (checked) and 'Assume referential integrity' (unchecked). The second column is titled 'Cross filter direction' and contains a dropdown menu set to 'Both'. Below it is a checkbox 'Apply security filter in both directions' (unchecked).

Power BI typically sets these options automatically and you won't need to adjust them; however, there are several situations where you might want to configure these options yourself.

Automatic relationship updates

You can manage how Power BI treats and automatically adjusts relationships in your reports and models. To specify how Power BI handles relationships options, select **File** > **Options and settings** > **Options** from Power BI Desktop, and then select **Data Load** in the left pane, under the *Current File* section at the bottom. The options for **Relationships** appear.



There are three options that can be selected and enabled:

- **Import relationships from data sources on first load:** This option is selected by default. When it's selected, Power BI checks for relationships defined in your data source. If such relationships exist, they are mirrored into the Power BI data model when you initially load data. This option enables you to quickly begin working with your model, rather than requiring you find or define those relationships yourself.
- **Update or delete relationships when refreshing data:** This option is unselected by default. If you select it, Power BI checks for changes in data source relationships when your dataset is refreshed. If those relationships changed or are removed, Power BI mirrors those changes in its own data model, updating or deleting them to match.
- **Autodetect new relationships after data is loaded:** With this option, Power BI will look for possible relationships between the tables uploaded in the model and it will create the relationships for you.

Leave these options as they are for now. But you know where to go, if you need to change any of them later on. Click Cancel to close the window.

Step 2 Future updates to the data require a different cardinality (from one-to-many to many-to-many)

Normally, Power BI Desktop can automatically determine the best cardinality for the relationship (for example one-to-one or one-to-many). If you do need to override the automatic settings, because you know the data will change in the future, you can change it with the **Cardinality** control. In this section, we will look at an example where we need to select a different cardinality.

For this step we will use the **CompanyProject** table and the **ProjectBudget** table. This last table is the set of projects for which a budget has been approved.

	A ^B _C ProjectName	A ^B _C Priority
1	Blue	A
2	Red	B
3	Orange	C
4	Purple	B
5	Yellow	C
6	Green	C

CompanyProject Table

	A ^B _C ProjectName	1 ² ₃ BudgetAllocation	AllocationDate
1	Blue	40000	12/01/2012
2	Red	100000	12/01/2012
3	Green	50000	12/01/2012

ProjectBudget Table. Set of projects for which a budget has been approved

Let's create a relationship between the **ProjectName** column in the **ProjectBudget** table and the **ProjectName** column in the **CompanyProject** table. To do so, click on Manage Relationships from the Home ribbon, then click on New to create a new relationship. Fill in the Creating Relationships window as shown below. Don't forget to select the columns you want to connect by clicking on them. The final window should look like the one below.

Create relationship

Select tables and columns that are related.

ProjectBudget

ProjectName	BudgetAllocation	AllocationDate
Blue	40000	12 January 2012
Red	100000	12 January 2012
Green	50000	12 January 2012

CompanyProject

ProjectName	Priority
Blue	A
Red	B
Orange	C

Cardinality

One to one (1:1)

Cross filter direction

Both

☒ Make this relationship active

☐ Apply security filter in both directions

☐ Assume referential integrity

OK

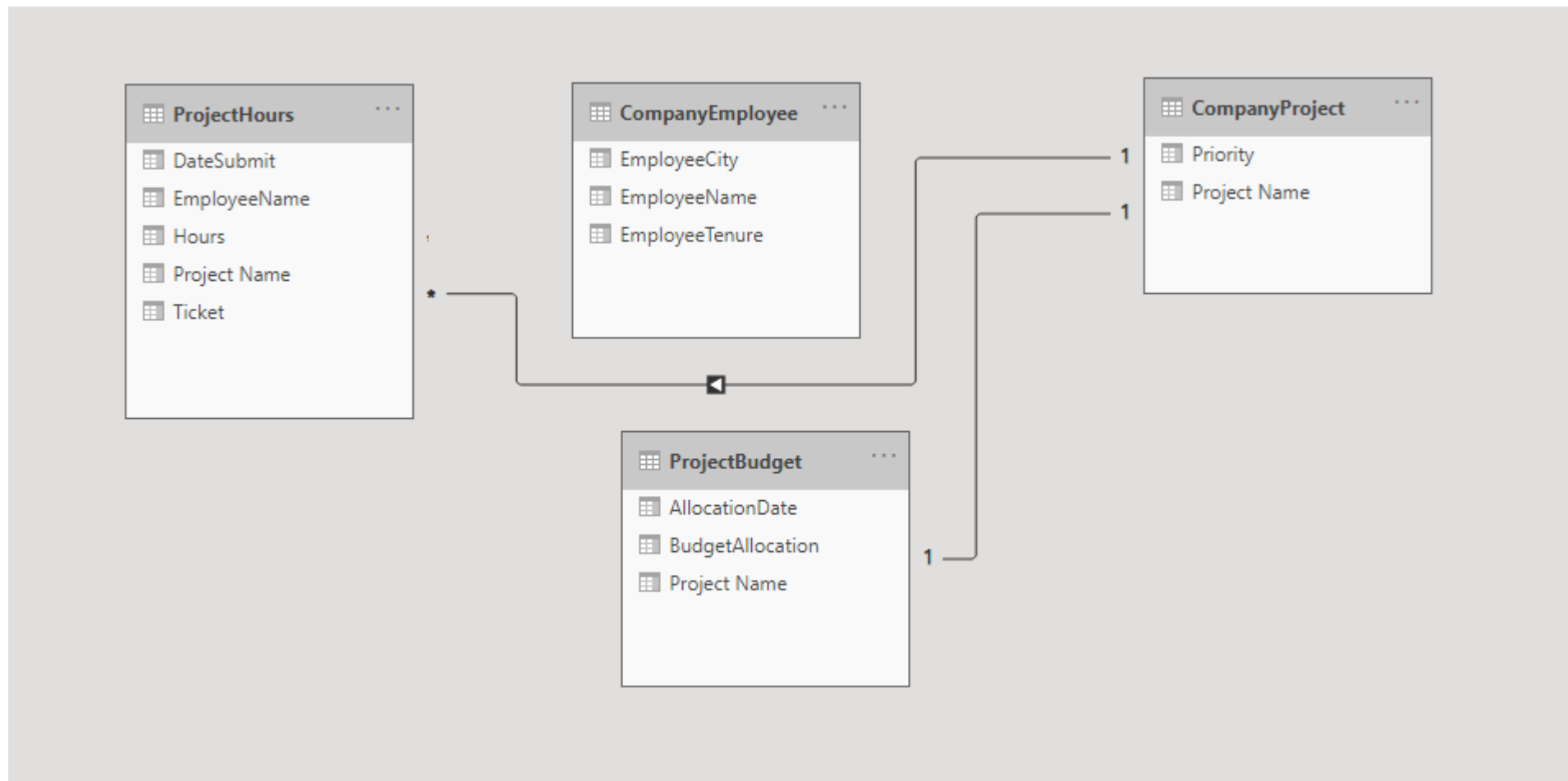
Cancel

After you select the columns, Power BI automatically sets **Cardinality** to **One to one (1:1)** and **Cross filter direction** to **Both**.

The reason Power BI makes these settings is because, to Power BI Desktop, the best combination of the two tables is as follows:

ProjName	Priority	BudgetAllocation	AllocationDate
Blue	A	40,000	12/1/2012
Red	B	100,000	12/1/2012
Green	C	50,000	12/1/2012
Yellow	C		
Purple	B		
Orange	C		

There's a one-to-one relationship between our two tables because there are no repeating values in the combined table's **ProjectName** column. The **ProjectName** column is unique, because each value occurs only once; therefore, the rows from the two tables can be combined directly without any duplication. Click OK to close the Creating Relationship window and Close the ManageRelationship window. You should now see the new relationship in the model view as shown below.



The "1"s at both ends of the link tell you that this relationship is a one-to-one relationship.

If you know that the data will change the next time you refresh it, this relationship might not be a one-to-one anymore, as shown in the example below. Let's suppose that a refreshed version of the **ProjectBudget** table now has additional rows for the Blue and Red projects:

Approved Projects	BudgetAllocation	AllocationDate
Blue	40,000	12/1/2012
Red	100,000	12/1/2012
Green	50,000	12/1/2012
Blue	80,000	6/1/2013
Red	90,000	6/1/2013

These additional rows mean the best combination of the two tables now looks like this:

In this new combined table, the **ProjectName** column has repeating values. The two original tables will not have a one-to-one relationship once the table is refreshed. In this case, because we know those future updates will cause the **ProjectName** column to have duplicates, we want to set the **Cardinality** to be **Many to one (*:1)**, with the *many* side on **ProjectBudget** and the *one* side on **CompanyProject**.

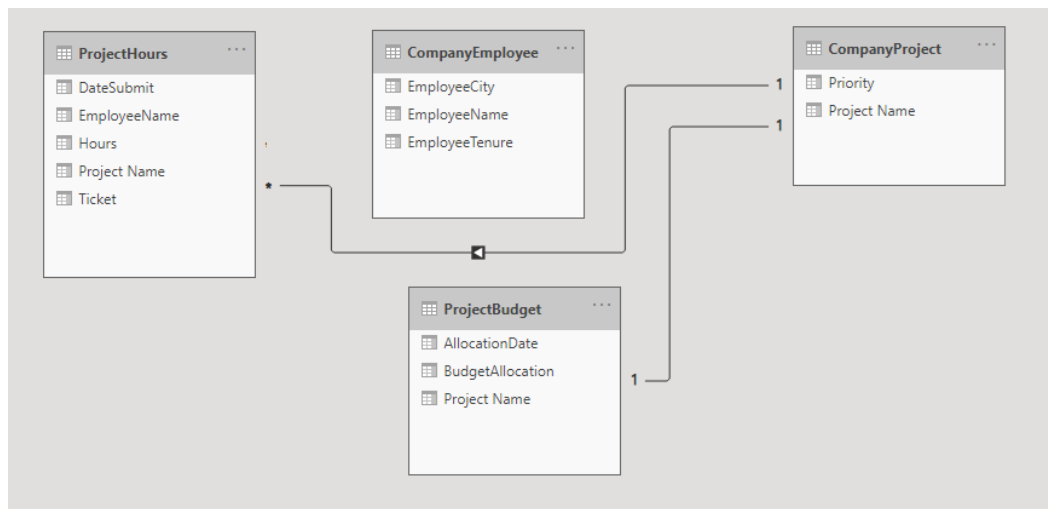
This is an example of a one-to-one relationship that can change into Many-to-one once more data comes in.

ProjName	Priority	BudgetAllocation	AllocationDate
Blue	A	40,000	12/1/2012
Red	B	100,000	12/1/2012
Green	C	50,000	12/1/2012
Yellow	C		
Purple	B		
Orange	C		
Blue	A	80000	6/1/2013
Red	B	90000	6/1/2013

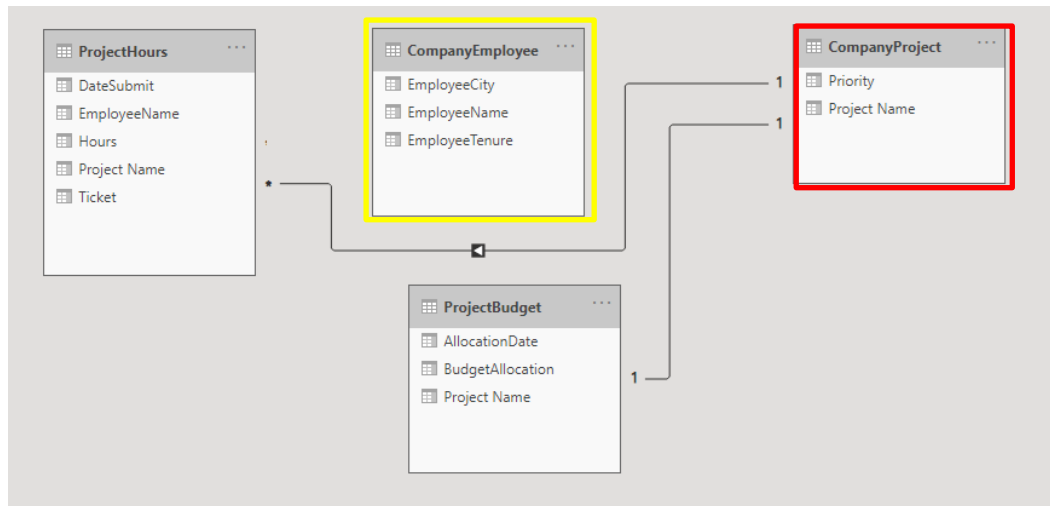
Step 3: Adjusting Cross filter direction for a complex set of tables and relationships

For most relationships, the cross filter direction is set to **Both**. There are, however, some more uncommon circumstances where you might need to set this option differently from the default, like if you are importing a model from an older version of Power Pivot, where every relationship is set to a single direction.

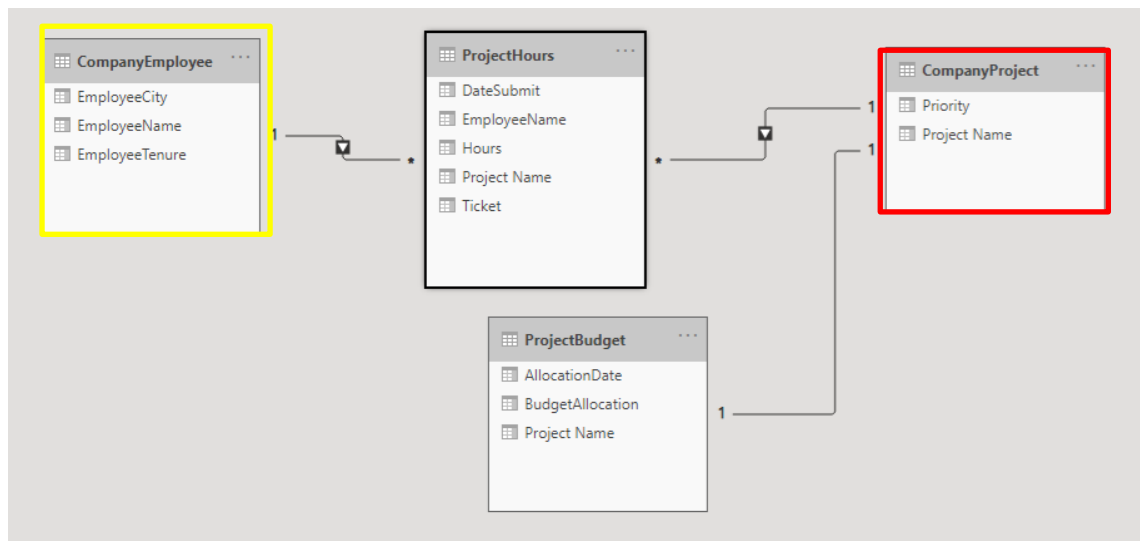
The image below shows the model so far. There is no link yet between the **Project Hours** and the **CompanyEmployee** tables.



The tables **ProjectHours** and **CompanyEmployees** can be connected using the EmployeeName column (from the ProjectHours table, this column reports the name of the Employee who has submitted the project) and the EmployeeName column (from the CompanyEmployee table). If you look at the content of both tables, you will see that they contain the same type of values. For this reason, they can be connected. To create the relationship between these tables, we will use a quicker way than what done so far. Click on the EmployeeName field from the CompanyEmployee table, drag and drop it on the EmployeeName field in the ProjectHours table. The resulting model is shown below:



Click on the Tables' headers to rearrange the tables as shown below.

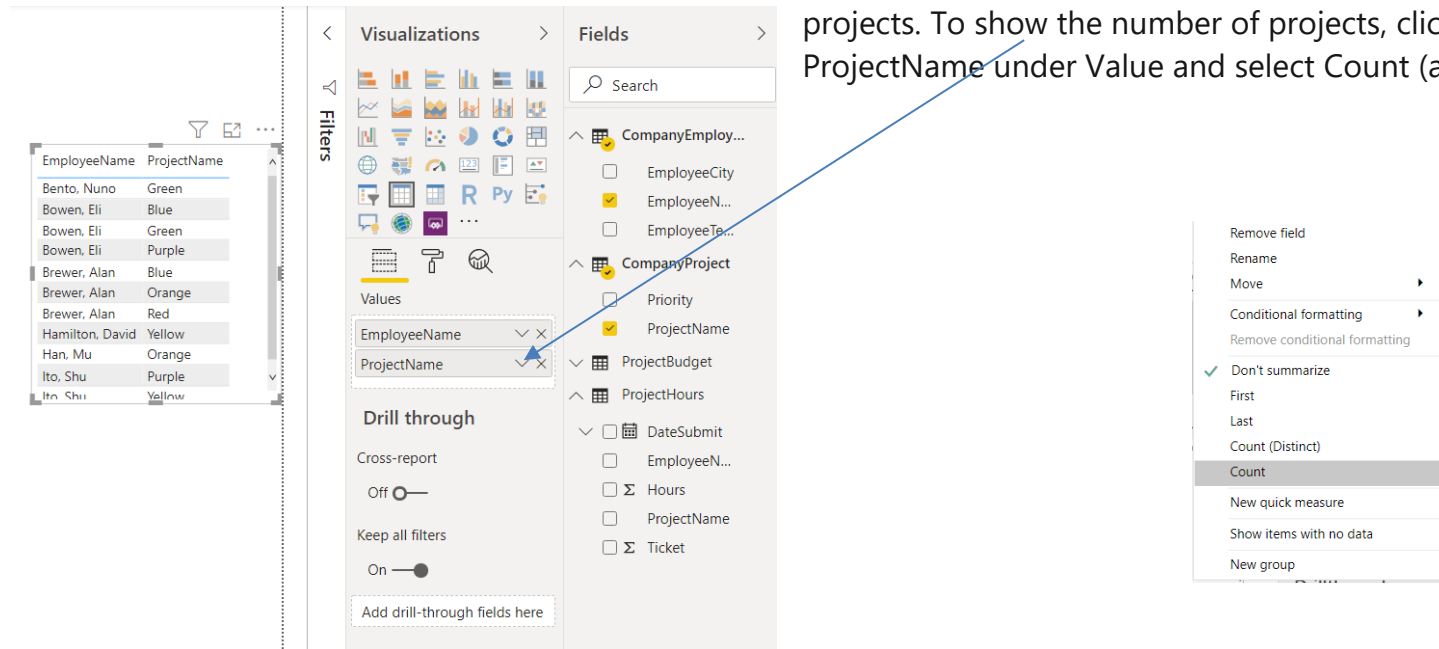


Please note that the **CompanyEmployee** and **CompanyProject** tables are now connected through the **ProjectHours** table. We are going to use this relationship to create a table. Be careful with the direction of the relationships, this will affect the resulting report.

Now let's use the relationships created so far, to create a report-table.

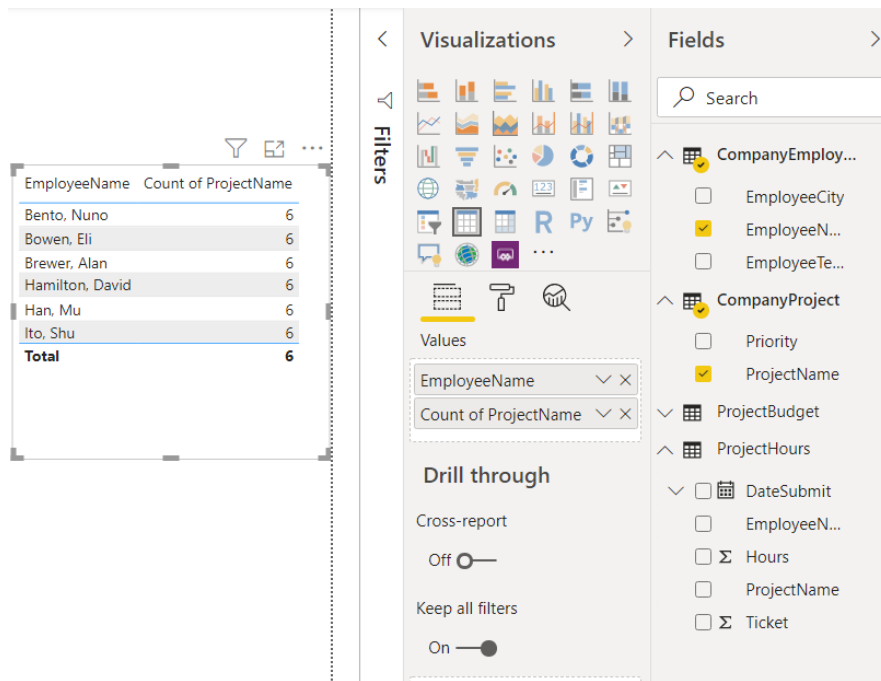
Open the Report view, and make sure the previous table is not selected. You can unselect the previous table by clicking anywhere in the blank canvas of the report view. Tick the EmployeeName field from the **CompanyEmployee** table and the ProjectName field from the **CompanyProject** table, as show below. We want the number of projects per Employee, but the table is currently showing the Name of the

projects. To show the number of projects, click on the dropdown arrow in the ProjectName under Value and select Count (as shown below).



The screenshot shows the Power BI interface. On the left, a table view displays data with columns 'EmployeeName' and 'ProjectName'. The 'Fields' pane on the right shows the 'CompanyEmployee' and 'CompanyProject' tables. 'EmployeeName' is selected from 'CompanyEmployee' and 'ProjectName' is selected from 'CompanyProject'. A dropdown arrow next to 'ProjectName' in the 'Values' section is clicked, opening a menu with options: 'Remove field', 'Rename', 'Move', 'Conditional formatting', 'Remove conditional formatting', 'Don't summarize', 'First', 'Last', 'Count (Distinct)', 'Count', 'New quick measure', 'Show items with no data', and 'New group'. The 'Count' option is highlighted.

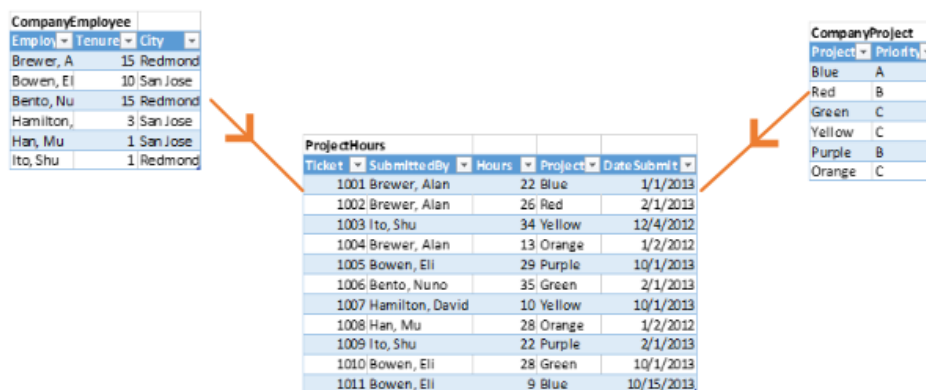
You should now see a different table:



This table shows the number of projects per employee. In the table, you are using the relation between the **CompanyEmployee** and **CompanyProject** tables. You can do this because they are connected through the **ProjectHours** table (as seen before).

However, there seems to be something wrong with the resulting table...The Count of Project Name column is reporting the wrong numbers.

This is because both the relationship's cross filtering direction is set to a single direction: towards the **ProjectHours** table:

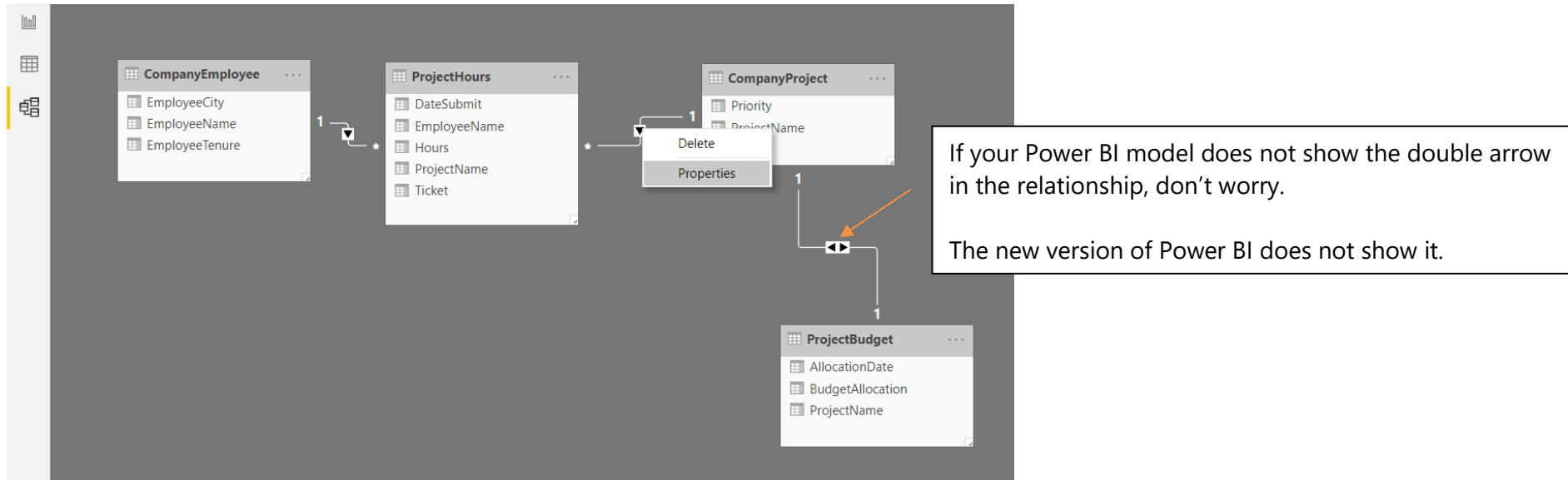


For this reason, the filter can be applied only from the external tables toward the middle one. But you can't go from one external table to the other one.

Filter specification will flow from the **CompanyProject** to **CompanyEmployee** (as shown on the right), but it won't flow up to **CompanyEmployee**.

However, if you set the cross filtering direction to **Both**, it will work. The **Both** setting allows the filter specification to flow up to **CompanyEmployee**. Let's see how to do this.

Go back to model view and right-click on the relationship between the **CompanyProject** table and the **ProjectHours** table as shown below. (In the screenshot below, I've moved the ProjectBudget table to make clearer on which link you have to right-click). Then select Properties.



In the Edit Relationships window, change the Cross-Filter direction as Both and click OK.

Edit relationship

Select tables and columns that are related.

ProjectHours

Ticket	SubmittedBy	Hours	Project	DateSubmit
1001	Brewer, Alan	22	Blue	01 January 2013
1002	Brewer, Alan	26	Red	02 January 2013
1003	Ito, Shu	34	Yellow	12 April 2012

CompanyProject

ProjName	Priority
Blue	A
Red	B
Green	C

Cardinality

Many to one (*:1)

☒ Make this relationship active

☐ Assume referential integrity

Cross filter direction

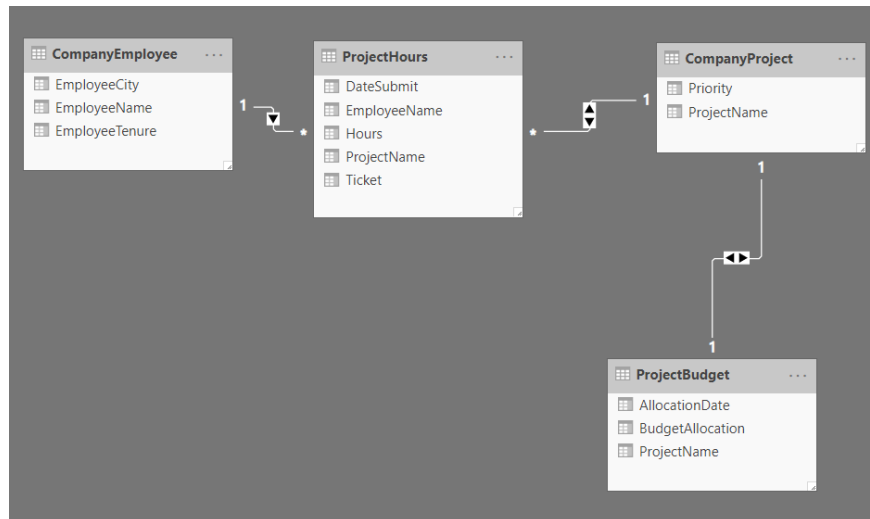
Single

Single

Both

OK

Cancel



You should now see the double-arrow in the relationship showing the Both-directional filter.

In this way the filter can “flow” from the CompanyEmployee table to the CompanyProject table through the ProjectHours table.

Let's go back to Report view now, the table is now showing the correct data.

EmployeeName	Count of ProjectName
Bento, Nuno	1
Bowen, Eli	3
Brewer, Alan	3
Hamilton, David	1
Han, Mu	1
Ito, Shu	2
Total	6

This has been possible only because we changed the relationship from Single to Both.

However, we have seen in lesson that if you have a much more complex model, Both-directional relationships can create problems.

There are several different ways you can handle this situation and we have seen a few in lesson. Here are the two most common:

- Delete or mark relationships as inactive to reduce ambiguity. Then, you might be able to set a relationship cross filtering as **Both**.
- Use the CROSSFILTER function to mark that relationship as a Both-directions one only for a specific measure.

There is a lot more about relationships, but you should now have an idea of how to use them and how they work in Power BI. We have seen the effects that wrong relationships can have on reporting data. Make sure that you always work with a well-structured model in order to avoid such mistakes.